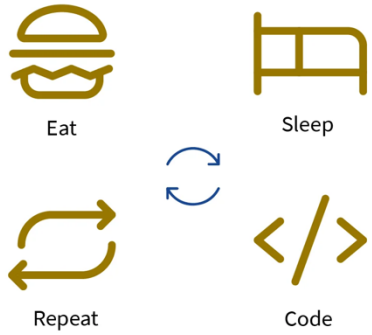
# CONTROL STATEMENTS

## Control Statement- Introduction

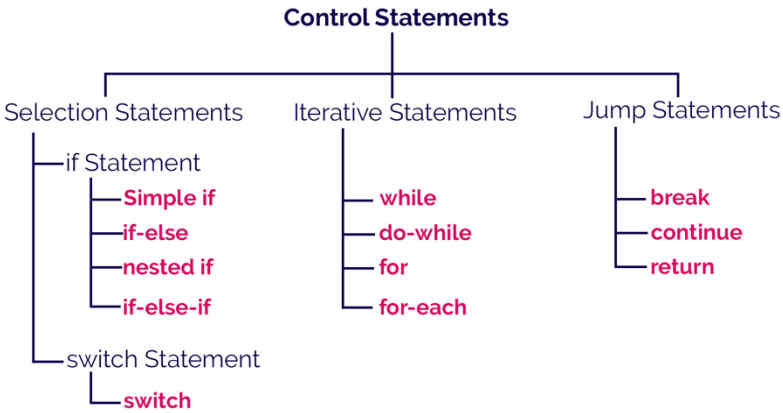
In the programs that we have examined to this point, each of the statements is executed once, in the order given. Most programs are more complicated because the sequence of statements and the number of times each is executed can vary.




Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides **statements** that can be used to control the flow of Java code. We use the term ***control flow*** to refer to statement sequencing in a program. It is one of the fundamental features of Java, which provides a smooth flow of program.

Java provides three types of control flow statements.

1. Selection Statements:
    - if Statements
    - if-else Statements
    - nested- if Statements
    - if-else if Statements
    - switch Statements
2. Iterative Statements:
    - do while Loop
    - while Loop
    - for Loop
    - for-each Loop
3. Jump Statements:
    - break Statements
    - continue Statements
    - return Statements



## Selction Statements

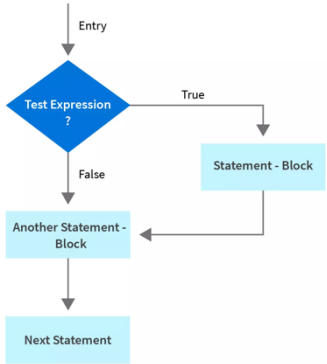
Also known as Decision making statement. As the name suggests, decision-making statements decide which statement to execute and when. Decision-making statements evaluate the Boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision-making statements in Java, i.e., if statement and switch statement.

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false. In Java, there are four types of if-statements given below.

1. if Statements
2. if-else Statements
3. if-else-if Ladder
4. nested if-Statement

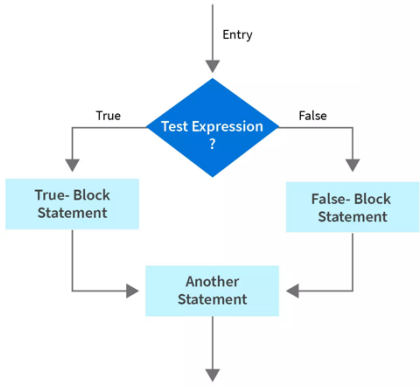
## if Statements

It is the most basic statement among all control flow statements in Java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.



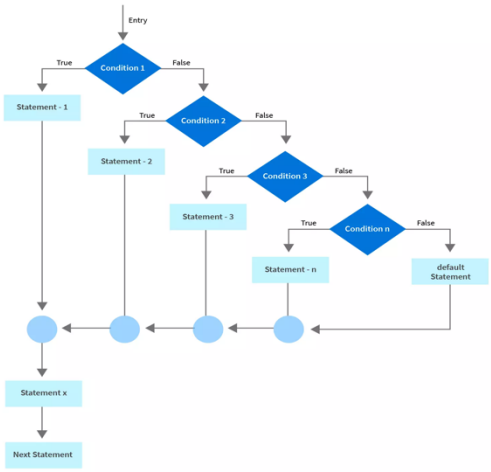

## if-else Statements

The if-else statements is an extension to the if-statement, which uses another block of code, i.e., else block. The else block is executed if the condition of the if-block is evaluated as false.



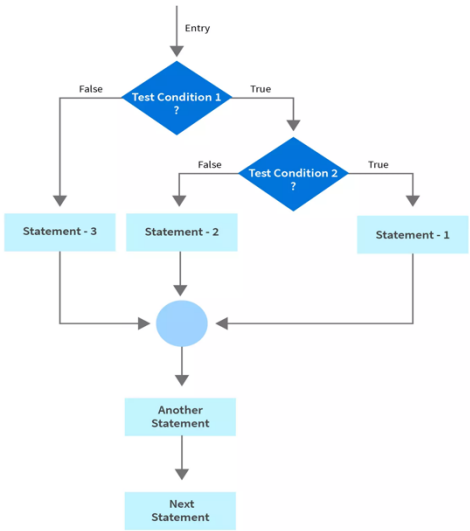
## if-else-if Ladder

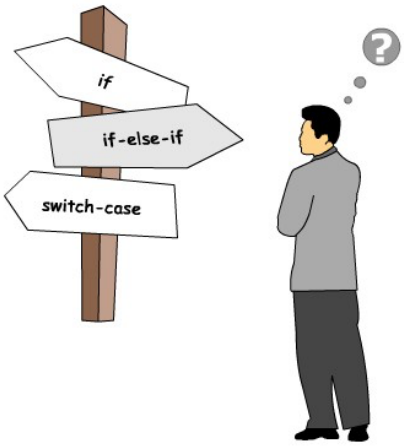
The if-else-if statement contains the if-statement followed by multiple else-if statements. In other words, we can say that it is the chain of if-else statements that create a decision tree where the program may enter in the block of code where the condition is true. We can also define an else statement at the end of the chain.




## nested-if Statements

In nested if-statements, the if statement can contain a **if** or **if-else** statement inside another if or else-if statement.

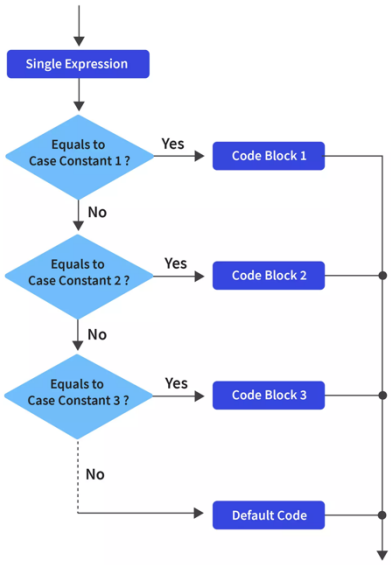




## switch Statements

In Java, switch Statements are similar to if-else-if statements. The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched. The switch statement is easier to use instead of if-else-if statements. It also enhances the readability of the program.

Points to be noted about switch statement:




- The case variables can be int, short, byte, char, or enumeration. String type is also supported since version 7 of Java.
- Cases cannot be duplicate
- Default statement is executed when any of the case doesn't match the value of expression. It is optional.
- While using switch statements, we must notice that the case expression will be of the same type as the variable. However, it will also be a constant value.
- Break Statement terminates the switch block when the condition is satisfied. It is optional, but if we use what's the difference between switch statement and if-else if statement. If we don't use we can execute the satisfied condition block and the block below it. This programme can be used in music players.

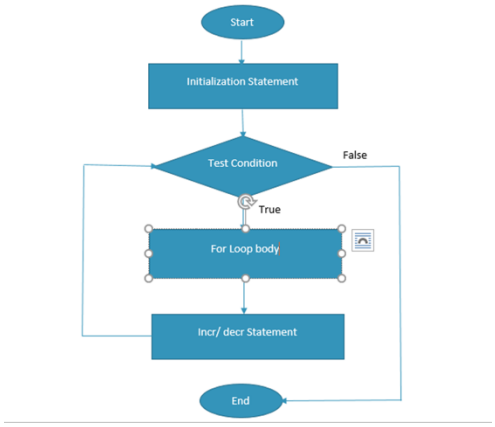
## Iterative Statements

In programming, sometimes we need to execute the block of code repeatedly while some condition evaluates to true. However, loop statements are used to execute the set of instructions in a repeated order. The execution of the set of instructions depends upon a particular condition.
In Java, we have three types of loops that execute similarly. However, there are differences in their syntax and condition checking time. Let's understand the loop statements one by one.

1. for Loop
2. while Loop
3. do-while Loop

## for Loop

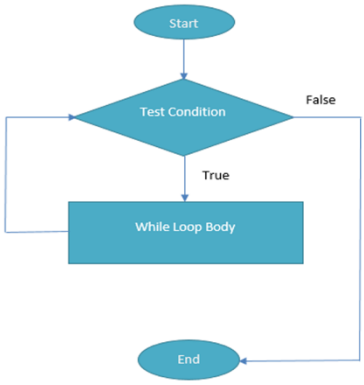
It enables us to initialize the loop variable, check the condition, and increment/decrement in a single line of code. We use the for loop only when we exactly know the number of times, we want to execute the block of code.




## while Loop

The while Loop is also used to iterate over the number of statements multiple times. However, if we don't know the number of iterations in advance, it is recommended to use a while loop. Unlike for loop, the initialization and increment/decrement doesn't take place inside the loop statement in while loop.

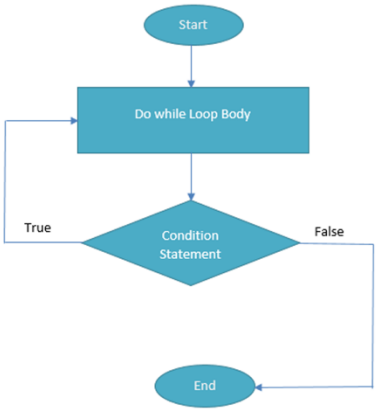
It is also known as the entry-controlled loop since the condition is checked at the start of the loop. If the condition is true, then the loop body will be executed; otherwise, the statements after the loop will be executed.




## do-while Loop

The do-while Loops checks the condition at the end of the loop after executing the loop statements. When the number of iteration is not known and we have to execute the loop at least once, we can use do-while loop.

It is also known as the exit-controlled loop since the condition is not checked in advance.



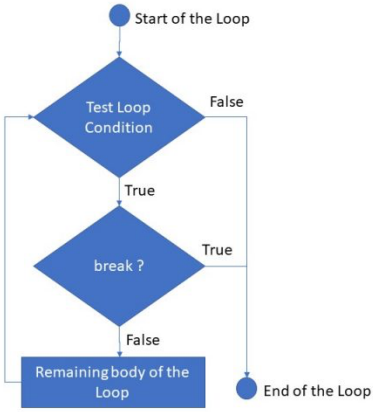
## Jump Statements

Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program.

There are three types of jump statements in Java. Let's understand the jump statements one by one.

1. break Statement
2. continue Statement
3. return Statement

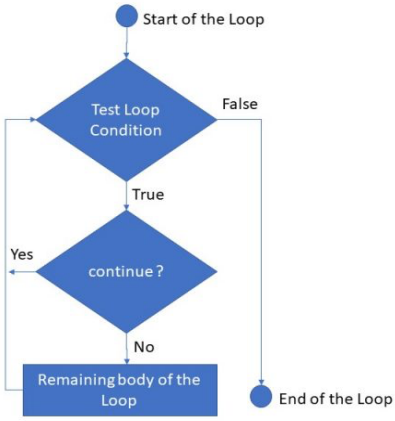
## break Statements




As the name suggests, the break Statement is used to break the current flow of the program and transfer the control to the next statement outside a loop or switch statement. However, it breaks only the inner loop in the case of the nested loop.

The break statement cannot be used independently in the Java program, i.e., it can only be written inside the loop or switch statement.

## continue Statements




Unlike break statement, the continue statement doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.
Be remember in while loop when we come out of loop body we come to condition check, while in for loop we come to inc/dec. So if we want to inc/dec then we have to do it in continue block in while loop.

## return Statements

In Java programming, the return statement is used for returning a value when the execution of the block is completed. The return statement inside a loop will cause the loop to break and further statements will be ignored by the compiler.

*Returning a value from a method will be discussed later in Methods Chapter.*